

L4 中短消息的 IPC 数据通信方式类似，LRPC 在通信调用的过程中不会切换通用寄存器，而是直接使用当前的通用寄存器。

寄存器和参数栈一起构成了 LRPC 的通信设计。客户端进程会优先使用寄存器，在寄存器不够的情况下则用参数栈传递参数。

通信连接的建立

内核需要为通信的服务端提供一个服务的抽象，即服务描述符。所有支持客户端调用的服务端进程将自己的处理函数等信息注册到服务描述符中。在系统内核里，需要为每个服务描述符准备两个资源：第一是参数栈，第二是连接记录（Linkage Record）。参数栈我们已经介绍过了，它将被同时映射到调用者进程和被调用者进程。而连接记录主要是记录调用过程的信息，类似于函数调用中往栈上压入的返回地址等。当一个服务被调用，服务端进程执行完请求需要返回时，内核会从连接记录中获得返回地址等信息。当客户端申请和服务端建立连接时，内核会分配参数栈和连接记录，并返回给客户端进程一个绑定对象（Binding Object），后续客户端可以通过绑定对象发起通信。绑定对象的获得意味着客户端和服务端成功建立了连接。

通信过程

当调用者发起一次通信时：内核验证绑定对象的正确性，并找到正确的服务描述符；内核验证参数栈和连接记录的正确性；内核检查是否有并发调用（可能导致参数栈等异常）；内核将调用者的返回地址和栈指针放到连接记录中；内核将连接记录放到线程控制结构体中的栈上（支持嵌套 LRPC 调用）；内核切换到被调用者进程地址空间；内核找到被调用者进程的运行栈（执行代码所使用的栈）；内核将当前线程的栈指针设置为被调用者进程的运行栈地址；内核将代码指针指向被调用者地址空间中的处理函数。从上面的过程中，我们可以看到 LRPC 设计将页表、运行栈以及代码指针视为负责处理请求的状态。对应的伪代码如代码片段 8.15 所示。

代码片段 8.15 LRPC 的伪代码

```

1 int ipc_call(A_stack, ...)           // A_stack 为参数栈
2 {
3     verify_binding(A_stack);          // 验证参数栈的正确性
4     service_descriptor = get_desc_from_A(A_stack);
5     // 其他安全检查：是否存在并发调用？
6     ...
7     save_ctx_to_linkage_record();      // 将调用信息保存到连接记录上
8     save_linkage_record();
9     ...
10    // 切换运行状态
11    switch_PT();                       // 修改页表
12    switch_cap_table(); // 修改权限表
13    switch_sp();                       // 修改栈地址
14    ...
15    // 返回用户态（服务端进程），不修改参数寄存器
16    ctx_restore_with_args(ret);

```

```
17 ...  
18 }
```

值得注意的是，不同的系统设计中需要考虑的要切换的状态是不同的。例如，代码片段 8.15 中我们将权限表也作为要切换的关键状态之一。

LRPC 中使用的迁移线程模型和 L4 中的直接进程切换有相似的地方，如它们都选择了绕开内核调度。不同点在于，L4 仍然完成了两个线程切换的任务，而在 LRPC 中其实没有切换线程。这在多核场景下的跨核进程间通信中有显著的不同：LRPC 在通信时可以将远端核上的被调用者上下文拉到当前核的线程上直接执行，相当于将跨核通信变成了单核通信；而 L4 却需要核间中断（IPI）等机制来通知远端核进行处理。

微内核进程间通信与宏内核进程间通信

目前介绍的 IPC 设计中既包括宏内核场景下的方案也包括微内核场景下的方案。由于宏内核和微内核的系统设计不同以及对进程间通信的要求不同，其设计通常存在一些差异。

首先，宏内核和微内核在进程设计上的差别会导致 IPC 设计的不同。相比宏内核，微内核架构中的进程既能够表示一个用户态的程序，也可以表示一个系统服务（如文件系统）。这需要微内核 IPC 机制引入用户态的命名服务等机制来管理进程和系统服务间的连接。而宏内核下的命名服务相对简单，是由内核自己完成的，如 Linux 的命名管道依赖文件名来实现命名服务的效果。后续章节将介绍的 Binder IPC 通信机制通常被认为借鉴了微内核的设计，其中一个原因就是 Android 存在大量的用户态系统服务，需要较为复杂的 IPC 管理机制。

其次，宏内核和微内核架构对于 IPC 的性能需求是不一样的，这同样影响了两类架构下的 IPC 设计。相比宏内核架构，微内核架构下的 IPC 频率显著增多，这要求 L4/seL4 等微内核系统采用更精巧的方式来优化 IPC 开销，如在切换上下文过程中只切换一个上下文的“最小集”。

最后，宏内核和微内核架构在内核的复杂性上也影响了各自的 IPC 设计。相比微内核的简洁，宏内核在内核层面包含大量的系统服务、驱动等代码。系统设计者通常很难在这样一个复杂的系统中引入一条 IPC 的快速路径。

8.7 案例分析：ChCore 进程间通信机制

本节主要知识点

□ ChCore 是如何设计和实现进程间通信机制的？

本节介绍与本书配套的实验内核 ChCore 中的进程间通信机制的设计。在 ChCore 的通信机制中，消息的传递和通知基于 LRPC 中的迁移线程技术和 L4 系列微内核中的直接进程切换技术，而数据的传输则是基于灵活的用户态共享内存。

代码片段 8.16 给出的是 ChCore 进程间通信的用户态样例代码，主要包括客户端进程代码（第 21 ~ 32 行）和服务端进程代码（第 1 ~ 19 行）。可以看到服务端进程通过 `ipc_register_server` 向内核注册一个服务。这里的关键参数为服务的逻辑处理函数，即代码中的 `ipc_dispatcher`。当一个请求被发出后，服务端的逻辑处理函数被调用。其首先获取一个消息中数据的地址（第 6 行），并且在处理完逻辑后通过 `ipc_return` 将结果返回客户端进程。客户端进程通过 `ipc_register_client` 注册一个进行通信的结构体，这里其实是通过 ChCore 中的 Capability 子系统来完成客户端和服务端进程之间连接的建立。当一切准备工作就绪后，客户端进程通过 `ipc_call` 发起一次远程调用的通信，而参数会被放在 `ipc_msg` 中。消息结构体（`ipc_msg_t`）中既可以传递数据，也可以传递 Capability。因此，可以通过传递内存内核对象的 Capability 的方式，在两个进程间建立一个共享内存区域，用作大量数据的传输。

代码片段 8.16 ChCore 进程间通信的用户态样例代码

```

1 // 服务端：用户态 IPC 逻辑处理
2 void ipc_dispatcher(ipc_msg_t *ipc_msg)
3 {
4     u64 ret;
5     // 获取消息中的数据
6     char* data = ipc_get_msg_data(ipc_msg);
7     // 执行服务代码
8     ...
9     // 返回结果给客户端进程
10    ipc_return(ret);
11 }
12
13 // 服务端进程代码
14 void server()
15 {
16     ...
17    ipc_register_server(ipc_dispatcher);
18    ...
19 }
20
21 // 客户端进程代码
22 void client()
23 {
24     int server_process_cap;
25     ipc_struct_t client_ipc_struct;
26     ipc_msg_t *ipc_msg;
27     ...
28     // 获得服务端进程的权限

```

```

29 ipc_register_client(server_process_cap, &client_ipc_struct);
30 ...
31 // 发起 IPC
32 u64 ret = ipc_call(&client_ipc_struct, ipc_msg);
33 ...
34 }

```

在用户态的示例代码中，和进程间通信相关的关键路径上的两个系统调用接口是 `ipc_call` 和 `ipc_ret`，这对应着代码片段 8.17 中的两个 ChCore 系统调用 `sys_ipc_call` 和 `sys_ipc_return`。可以看到，在拿到内核中的通信对象（`conn`）之后，内核就通过 `thread_migrate_to_server` 接口将控制流直接转交给服务端进程（客户端进程是类似的）。这里的思路和 LRPC 以及 L4 中直接进程切换的思路类似，对于通信的情况绕开内核调度，直接完成上下文切换。

代码片段 8.17 ChCore 进程间通信的系统调用接口

```

1 u64 sys_ipc_call(u32 conn_cap, ipc_msg_t *ipc_msg)
2 {
3     // 获取 IPC 连接
4     struct ipc_connection *conn = get_connection(conn_cap);
5     // 处理 ipc_msg，如传递 Capability 等
6     ...
7     // 将控制流移交给服务进程，arg 为传递给服务端的参数
8     thread_migrate_to_server(conn, arg);
9
10    BUG("This function should never return!\n");
11    ...
12 }
13
14 void sys_ipc_return(u64 ret)
15 {
16     // 获取 IPC 连接
17     struct ipc_connection *conn = get_current_connection();
18     ...
19     // 控制流从服务进程转移回客户进程，ret 为返回给客户端的结果值
20     thread_migrate_to_client(conn, ret);
21
22    BUG("This function should never return!\n");
23 }

```

8.8 案例分析：Binder IPC

本节主要知识点

- ❑ Android 场景对于进程间通信有什么特殊的需求？
- ❑ Binder IPC 中的通信模型是如何设计的？